

NIT2112

Object Oriented Programming

Session 08

AI-Assisted Coding and Debugging

Acknowledgement of Country



Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

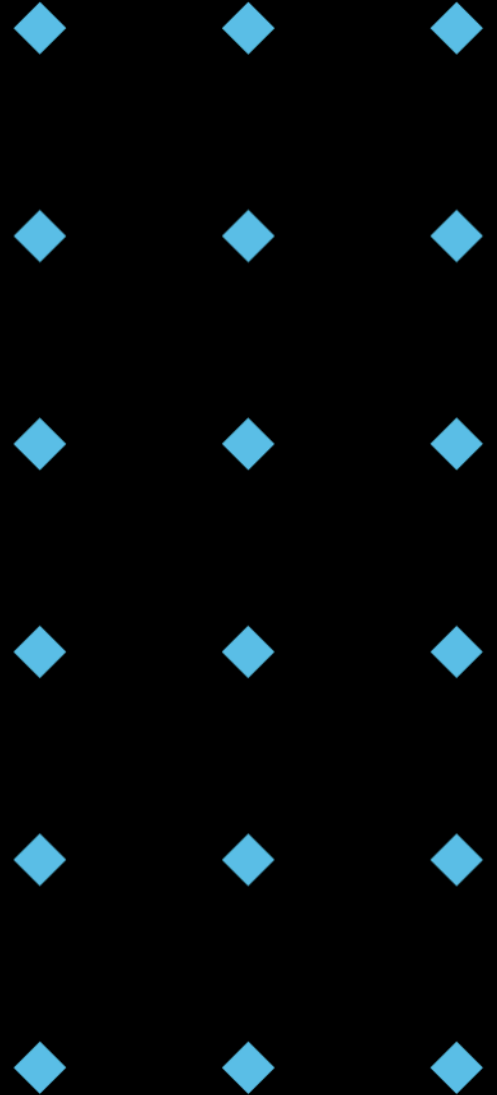
Session Outline

AI-Assisted Coding:

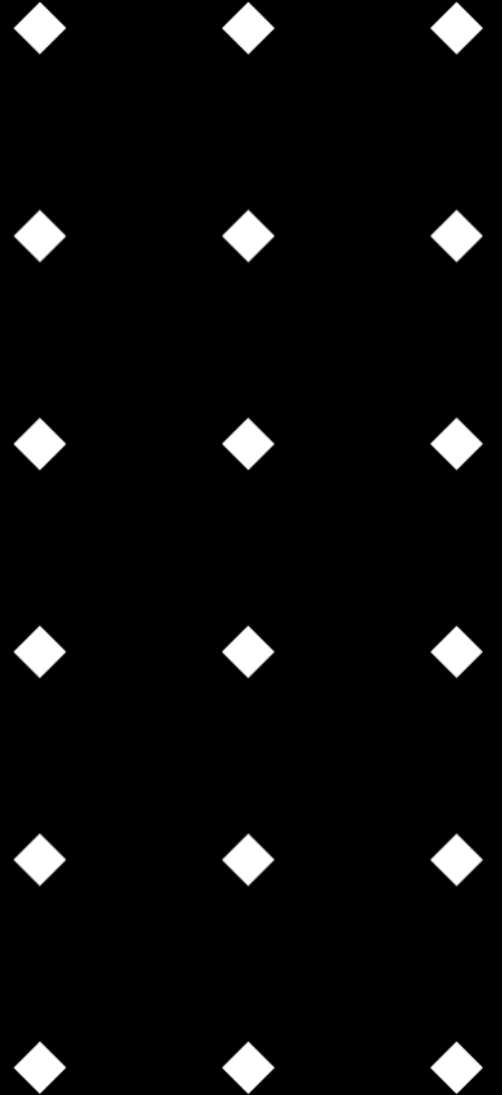
- ◆ Scaffolding Classes & Methods
- ◆ Generating Implementations
- ◆ Top-Down and Bottom-Up Design

AI-Assisted Debugging

- ◆ Error analysis.
- ◆ Runtime logic issues.
- ◆ Inheritance and polymorphism issues.

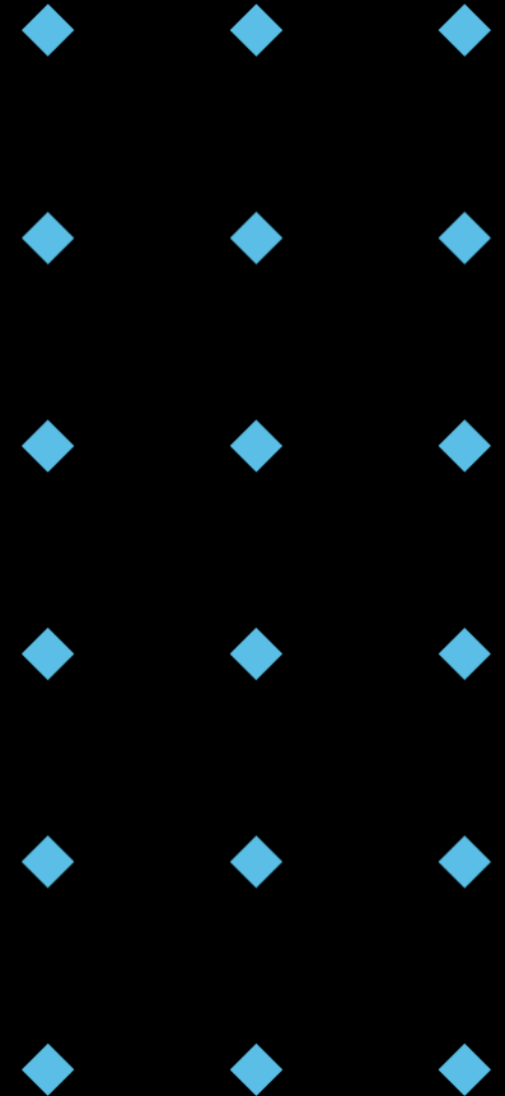


AI Assisted Coding



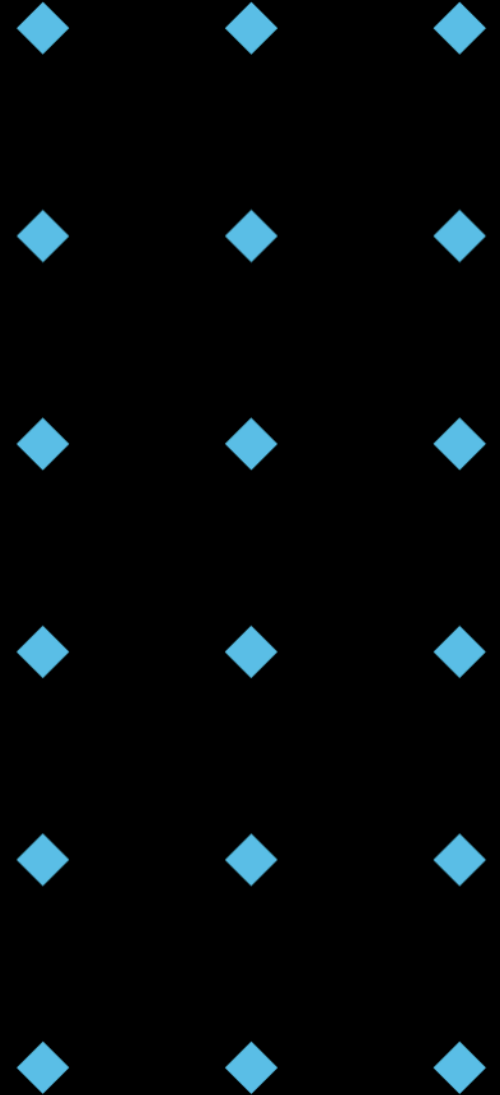
AI-Assisted OOP Coding

- ◆ Leveraging AI tools to **accelerate** and **enhance** the process of **designing**, **writing**, and **structuring** Object-Oriented code in Python.
- ◆ **Not** to replace the developer, but to augment their capabilities. Think of it as a powerful pair programmer.
- ◆ Benefits:
 - ◆ **Speed:** Faster generation of boilerplate code, methods, and structures.
 - ◆ **Consistency:** Can help adhere to specific coding styles or patterns.
 - ◆ **Exploration:** Quickly prototype different design ideas.
 - ◆ **Learning:** Can provide examples and explain concepts.
- ◆ Caution: AI-generated code ALWAYS requires review and understanding.



AI for Class Scaffolding

- ◆ **Scaffolding:** Creating the basic **structure** or **skeleton** of a class (or set of classes)
- ◆ Define the **class name**, **initial attributes** (`__init__`), and **method signatures** (names and parameters) without full implementation.
- ◆ How AI Helps: You provide a **description** or **requirements**, and the AI generates the initial class definition.



AI for Class Scaffolding

- ◆ Example Prompt (in comments or chat):

Create a Python class 'Student' with attributes for *name*, *student_id*, and *major*. Include an initializer (*__init__*) and methods to *get_details()* and *update_major(new_major)*.

- ◆ Potential AI Output:

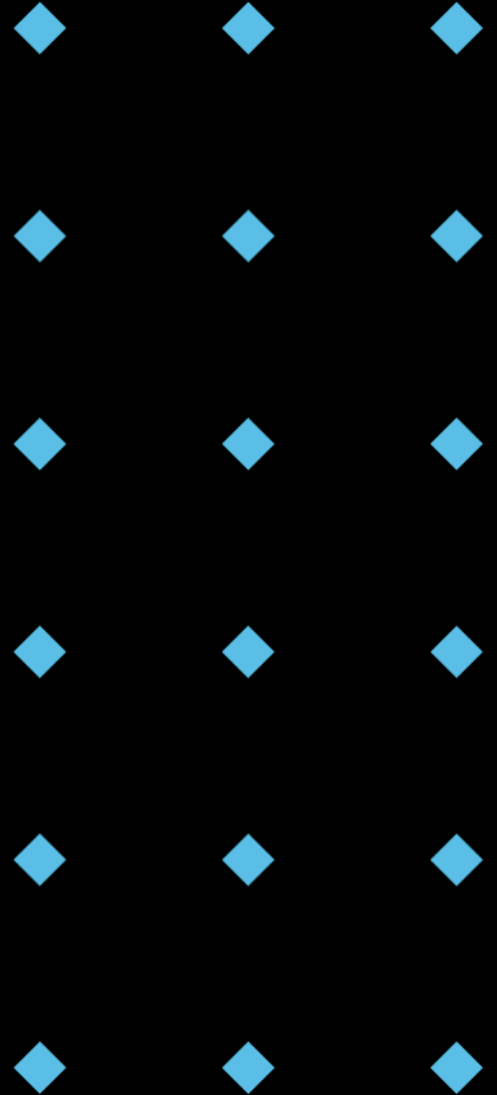


```
1 class Student:
2     """Represents a student with basic information."""
3
4     def __init__(self, name: str, student_id: str, major:
5         str):
6         """Initializes a new Student object."""
7         self.name = name
8         self.student_id = student_id
9         self.major = major
10        # Potential future attributes can be added here
11        self._enrollment_status = "Enrolled" # Example
12        of AI possibly adding more
13
14    def get_details(self) -> str:
15        """Returns a string with the student's details.
16        """
17
18        # TODO: Implement method logic
19        pass
20
21    def update_major(self, new_major: str) -> None:
22        """Updates the student's major."""
23        # TODO: Implement method logic
24        pass
25
26    # Note: AI might add type hints, docstrings, and basic
27    default logic.
```

Discussion Point

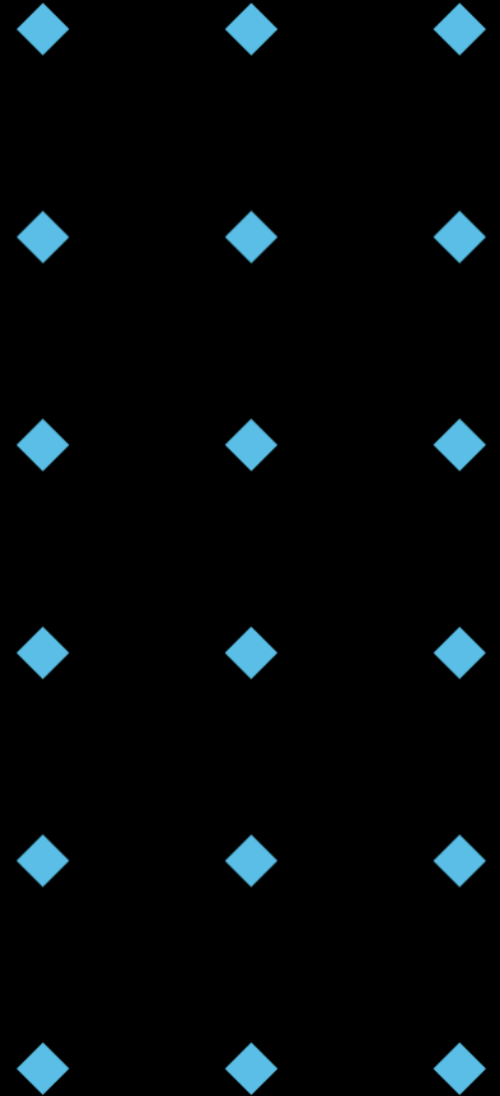
- ◆ Question: Imagine you need to create a class *Car* with attributes for *make*, *model*, *year*, and *current_speed*. It should have methods *accelerate(amount)*, *brake(amount)*, and *get_speed()*.
- ◆ How would you phrase a prompt for an AI Copilot to scaffold this class? What variations in the output might you expect?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.



AI for Method Implementation

- ◆ Once the class structure (scaffold) is in place, AI can help write the **actual code** inside the methods.
- ◆ Provide a clear **docstring** or **comment** explaining what the method should do.
- ◆ The AI uses this **description**, the **method signature**, and the **class context** to generate the implementation.



AI for Method Implementation

◆ Example Prompt:

```
1 class Student:
2     # ... (init method as before) ...
3
4     def get_details(self) -> str:
5         """Returns a formatted string containing the
6         student's name, ID, and major."""
7         # AI: Implement this method based on the
8         docstring and class attributes
9         pass
10
11    def update_major(self, new_major: str) -> None:
12        """Updates the student's major. Add validation?
13        """
14        # AI Prompt: Implement, perhaps add check for
15        valid major?
```

[Provide Code]
Please complete the *Student* class according to my
comments.

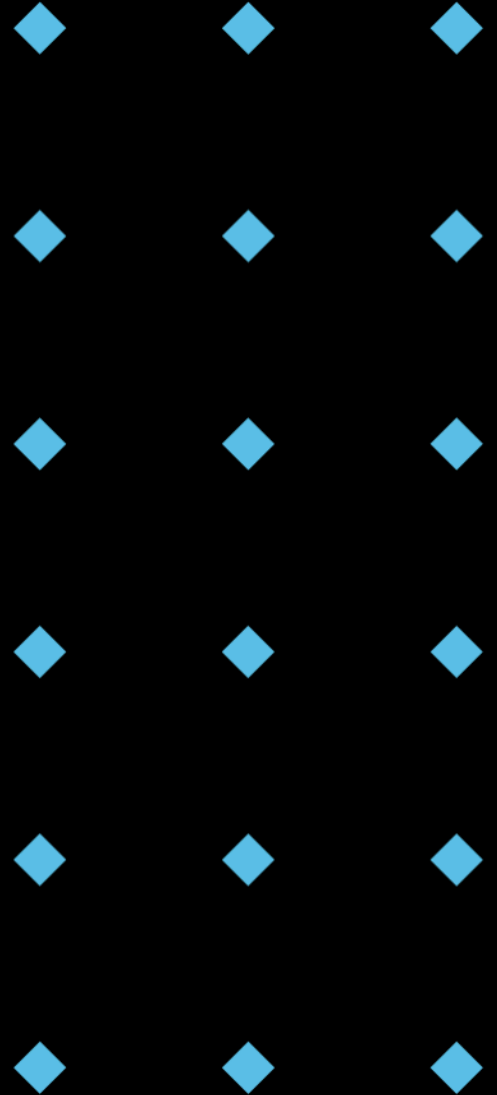
◆ Potential AI Output:

```
1 class Student:
2     # ... (init method as before) ...
3
4     def get_details(self) -> str:
5         """Returns a formatted string containing the
6         student's name, ID, and major."""
7         return f"Name: {self.name}, Student ID: {self.
8         student_id}, Major: {self.major}"
9
10    def update_major(self, new_major: str) -> None:
11        """Updates the student's major. Add validation?
12        """
13        if isinstance(new_major, str) and len(new_major)
14        > 0:
15            self.major = new_major
16            print(f"{self.name}'s major updated to {
17            new_major}.")
18        else:
19            print("Invalid major provided.")
```



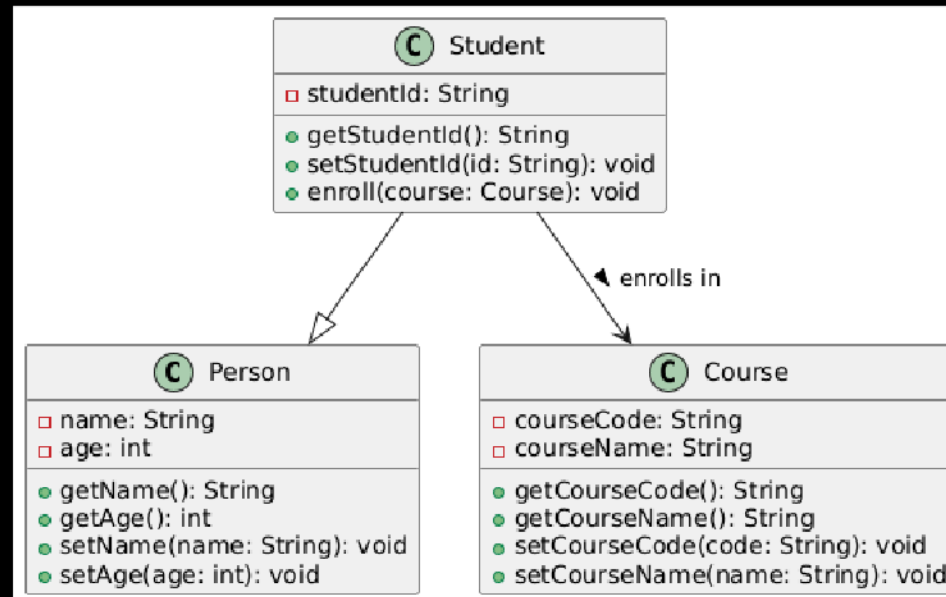
Quick Poll / Discussion

- ◆ Scenario: Consider the `Car` class again. For the `accelerate(amount)` method, the docstring is: "Increases the car's speed by the given amount, but not exceeding a max speed of 150."
- ◆ Question: What potential issues might arise if you blindly accept the AI-generated code for this method? What should you check?
 - ◆ A) Negative acceleration amount?
 - ◆ B) Data type of amount?
 - ◆ C) Handling of the max speed limit?
 - ◆ D) All of the above?



From Visual Design to Code: UML & AI

- ◆ Unified Modeling Language (UML) - a standard graphical notation for specifying, visualizing, constructing, and documenting software systems. Class diagrams are particularly relevant for OOP.
- ◆ AI's Role: Some AI tools (especially advanced chatbots or specialized tools) can interpret descriptions or even visual representations (if supported) of UML diagrams and generate corresponding Python code.



Prompting for UML Conversion

- ◆ Method 1: Textual Description (More Common for Current Copilots):
- ◆ Method 2: Direct Image/Diagram Input (Emerging Capability): Some tools might allow uploading or referencing a diagram directly (less common in basic copilots).
- ◆ Example Prompt:

Generate Python classes based on this UML description:
Class *Person*: attributes *name* (str), *age* (int). Method *introduce*().

Class *Student* inherits from *Person*: adds attribute *student_id* (str), *major* (str). Overrides *introduce*().

Class *Course*: attributes *course_code* (str), *title* (str), *credits* (int). Method *get_info*().

A *Student* can enroll in multiple *Courses* (association). Add an *enroll(course)* method to *Student* and store enrolled courses in a list.

◆ Potential AI Output:



```
1 class Person:
2     def __init__(self, name: str, age: int):
3         self.name = name
4         self.age = age
5     def introduce(self):
6         print(f"Hi, I'm {self.name}, {self.age} years old.")
7
8 class Student(Person): # Inheritance
9     def __init__(self, name: str, age: int, student_id: str, major:
10         str):
11         super().__init__(name, age)
12         self.student_id = student_id
13         self.major = major
14         self._enrolled_courses = [] # Association handled here
15
16     def introduce(self): # Overriding
17         super().introduce()
18         print(f"I am a student with ID {self.student_id}, majoring
19             in {self.major}.")
20
21     def enroll(self, course: 'Course'): # Type hint needs Course
22         defined
23         if isinstance(course, Course):
24             self._enrolled_courses.append(course)
25             print(f"{self.name} enrolled in {course.title}.")
26         else:
27             print("Invalid course object.")
28
29     def get_enrolled_courses(self):
30         return self._enrolled_courses
31
32 class Course:
33     def __init__(self, course_code: str, title: str, credits: int)
34         :
35         self.course_code = course_code
36         self.title = title
37         self.credits = credits
38
39     def get_info(self):
40         return f"{self.course_code}: {self.title} ({self.credits}
41             credits)"
```

Verifying AI-Generated UML Conversion

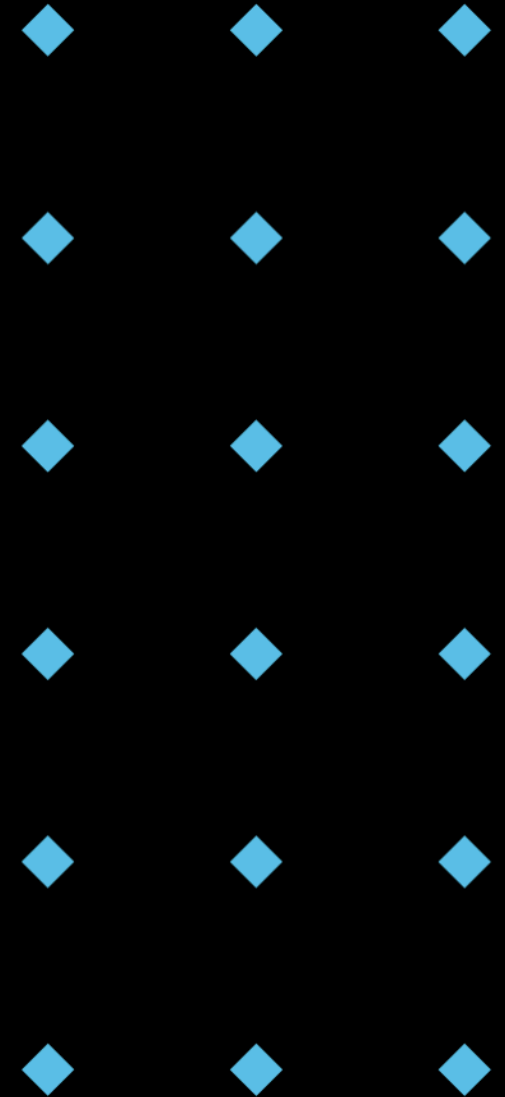
Key Checks:

- ◆ **Relationships:** Is inheritance correctly implemented (*super()* used)? Are associations represented appropriately (e.g., list of objects, dictionary)?
- ◆ **Attributes/Methods:** Are all specified attributes and methods present? Are data types correct (or inferred reasonably)?
- ◆ **Method Signatures:** Do method parameters match the UML?
- ◆ **Completeness:** Did the AI miss any details from the description or diagram?
- ◆ **Assumptions:** Did the AI make assumptions you didn't intend (e.g., adding extra methods or attributes)?

Remember: UML diagrams can be complex; AI might struggle with **intricate relationships**, **abstract classes**, or **interfaces** without very clear prompts.

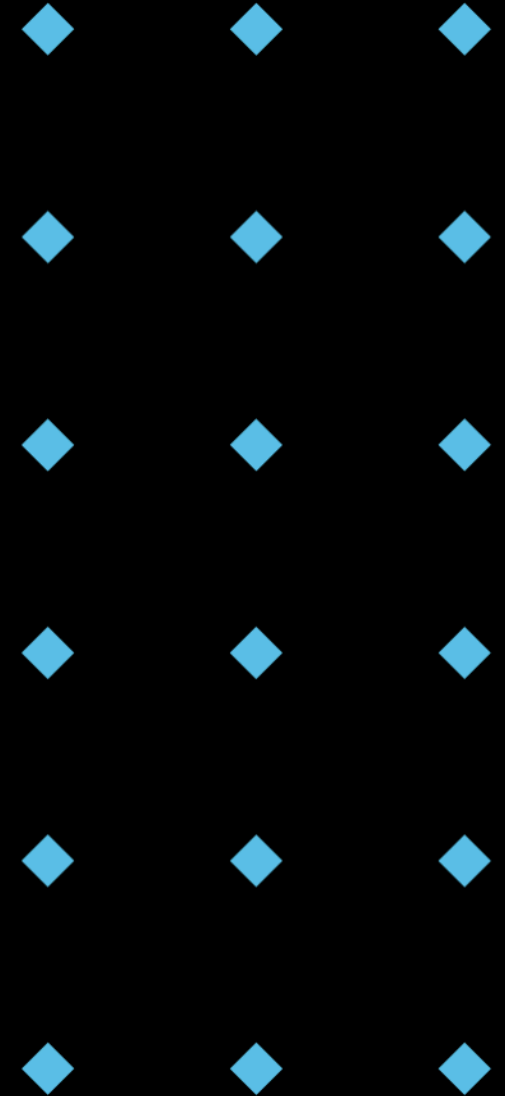
Top-Down and Bottom-Up Design

- ◆ **Top-Down Design:** Start with the **overall** system architecture and **high-level** components, then progressively refine and implement the details.
 - ◆ Identify major classes and their interactions first.
 - ◆ Define interfaces and abstract classes.
 - ◆ Gradually implement concrete subclasses and methods.
- ◆ **Bottom-Up Implementation:** Start by building **smaller, well-defined, concrete** components (classes or functions), then gradually combine them to form larger parts of the system.
 - ◆ Focus on implementing specific, low-level classes first.
 - ◆ Test these components individually.
 - ◆ Assemble them into more complex structures.



AI in Top-Down OOP Design

- ◆ How AI Helps:
 - ◆ **Brainstorming:** Ask AI for potential class structures based on requirements ("What classes might be needed for a library system?").
 - ◆ **Interface/Abstract Class Generation:** Generate skeletons for high-level concepts. ("Create an abstract class *Shape* with an abstract method *calculate_area()*").
 - ◆ **Scaffolding Subclasses:** Generate initial structures for concrete classes inheriting from abstract ones or implementing interfaces.



Top-Down AI Example: E-commerce System

- 1. High-Level Prompt:

Suggest main classes for a simple e-commerce system focusing on products, shopping carts, and orders. Show basic relationships.

- 3. Abstract Class Prompt:

Create an abstract class 'Product' using `abc` module, with attributes '`product_id`', '`name`', '`price`', and an abstract method '`get_product_type()`'.

- 2. Potential AI Output:

Product, ShoppingCart, Order, Customer.

Customer has a ShoppingCart, ShoppingCart contains Product items, Order is created from ShoppingCart.

- 4. Potential AI Output:

```
1  from abc import ABC, abstractmethod
2
3  class Product(ABC):
4      def __init__(self, product_id: int, name: str, price: float):
5          self.product_id = product_id
6          self.name = name
7          self.price = price
8
9      @abstractmethod
10     def get_product_type(self) -> str:
11         """Abstract method to return the type of product."""
12         pass
```

Top-Down AI Example: E-commerce System

◆ 5. Concrete Class Scaffolding Prompt:

Create a concrete class *'Book'* that inherits from *'Product'*. Add attributes *'author'* and *'isbn'*. Implement *'get_product_type()'*.

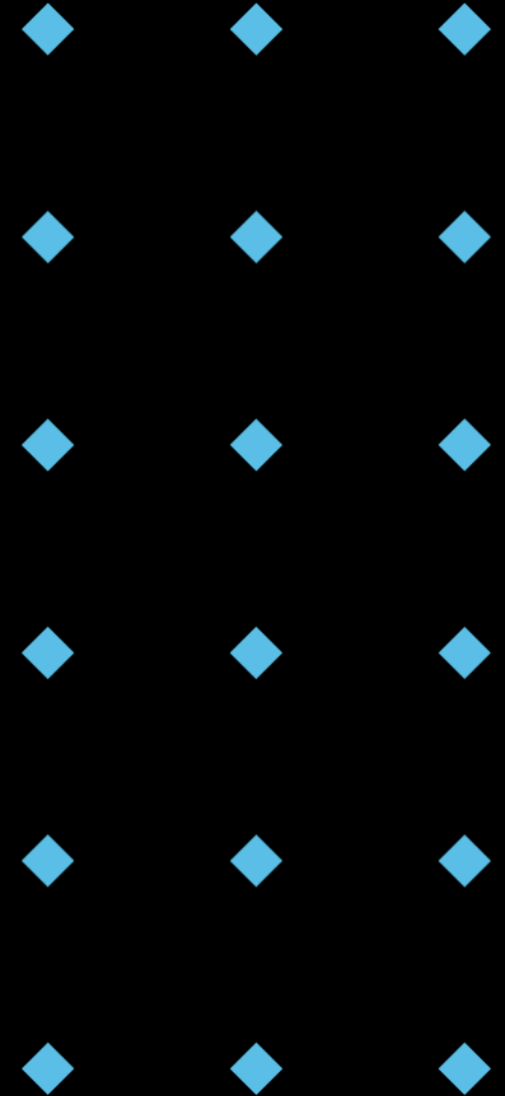
◆ 6. Potential AI Output:



```
1 class Book(Product):
2     def __init__(self, product_id: int, name: str, price: float,
3         author: str, isbn: str):
4         super().__init__(product_id, name, price)
5         self.author = author
6         self.isbn = isbn
7
8     def get_product_type(self) -> str:
9         """Return the type of product."""
10        return "Book"
11
12    def get_details(self) -> str:
13        """Return formatted book details."""
14        return (f"Book: {self.name}\n"
15            f"Author: {self.author}\n"
16            f"ISBN: {self.isbn}\n"
17            f"Price: ${self.price:.2f}")
```

AI in Bottom-Up OOP Implementation

- ◆ How AI Helps:
 - ◆ Implementing **Utility Classes**: Generate code for specific, reusable components (e.g., a *DataValidator* class, a *FileManager* class).
 - ◆ Generating **Specific Methods**: Implement algorithms or data manipulations within a class based on detailed requirements. ("Implement a method to sort a list of *Product* objects by price").
 - ◆ **Refactoring/Combining**: Suggest ways to **combine** smaller functions or classes into a more cohesive structure (Use with caution!).



Bottom-Up AI Example: Data Processing Tool

1. Implement Specific Component:

Create a Python class 'CSVReader' that takes a file path in `__init__`.
Add a method 'read_data()' that reads the CSV and returns a list of dictionaries (each dict represents a row, keys are header names). Handle `FileNotFoundError`.

2. Potential AI Output:



```
1 import csv
2
3 class CSVReader:
4     def __init__(self, file_path: str):
5         """Initialize the CSVReader with a file path."""
6         self.file_path = file_path
7
8     def read_data(self) -> list:
9         """Reads the CSV file and returns a list of dictionaries.
10         """
11         try:
12             with open(self.file_path, mode='r', encoding='utf-8')
13                 as file:
14                 csv_reader = csv.DictReader(file) # Uses first
15                 row as keys
16                 return [row for row in csv_reader] # Convert to
17                     list of dicts
18         except FileNotFoundError:
19             print(f"Error: File '{self.file_path}' not found.")
20             return []
```


Bottom-Up AI Example: Data Processing Tool

3. Implement Another Component:

Create a class `'DataProcessor'` with a method `'filter_by_value(data, column_name, value)'`. `'data'` is a list of dicts. The method should return a new list containing only dicts where the specified column matches the specified value.

5. Combine Components (Manual or AI-Assisted):

```
1 # Now, use CSVReader to get data, then pass it to DataProcessor
2 reader = CSVReader('input.csv')
3 try:
4     raw_data = reader.read_data()
5     processor = DataProcessor()
6     filtered_data = processor.filter_by_value(raw_data, 'City', '
7     print(filtered_data)
8 except FileNotFoundError:
9     print("Error: Input file not found.")
```

4. Potential AI Output:



```
1 class DataProcessor:
2     @staticmethod
3     def filter_by_value(data: list, column_name: str, value) ->
4         list:
5         """Filters a list of dictionaries by matching a column
6         value.
7
8         Args:
9             data (list): List of dictionaries (rows from a
10             dataset).
11             column_name (str): The key to filter by.
12             value: The value to match.
13
14         Returns:
15             list: Filtered list of dictionaries.
16
17         """
18         return [row for row in data if row.get(column_name) ==
19                 value]
```

Discussion: Design Approaches & AI

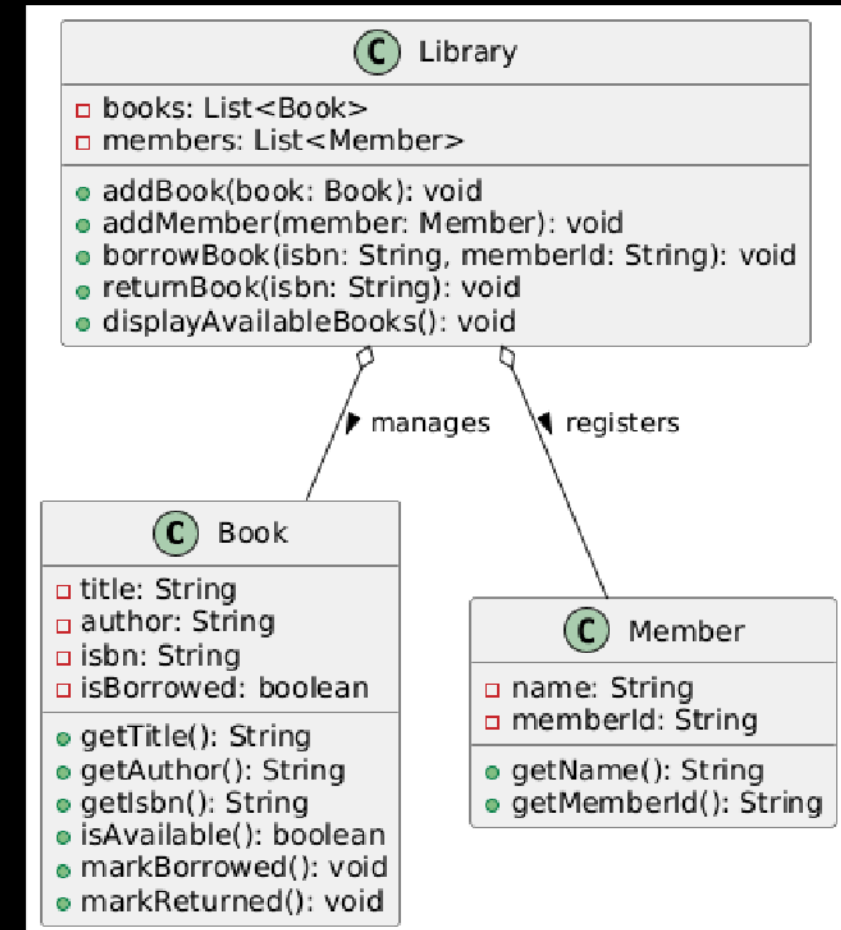
Question:

- ◆ When might a Top-Down approach with AI be more beneficial for an OOP project? (Think large systems, uncertain details).
- ◆ When might a Bottom-Up approach with AI be more suitable? (Think well-defined components, reusable utilities).
- ◆ Does using AI change your preference for one approach over the other? Why or why not?

* Instructions: Take 2 minutes to think individually, then discuss with a partner for 3 minutes.

Case Study: AI-Assisted Library System Build

- ◆ Goal: Build a basic command-line library system using Python OOP, assisted by an AI Copilot.
- ◆ Core Requirements:
 - ◆ Represent *Book* objects (*title*, *author*, *ISBN*).
 - ◆ Represent *Member* objects (*name*, *member ID*).
 - ◆ A *Library* class to manage books and members.
 - ◆ Functionality: Add book, add member, borrow book, return book, display available books.
- ◆ Approach: We will use a mix of **Top-Down** and **Bottom-Up**, leveraging AI at various stages.



Case Study: Step 1 - Class Design & Scaffolding

- ◆ Prompt (Top-Down aspect):

Design basic Python classes for a library system.

*Need classes for **Book** (title, author, isbn, is_borrowed flag), **Member** (name, member_id, borrowed_books list), and **Library** (manage collections of books and members).*

*The methods in **Library** include **add_book**, **add_member**, **borrow_book**(member_id, isbn), **return_book**(isbn), **display_available_books**.*

*Scaffold these classes with **__init__** methods and method signatures.*

- ◆ Potential AI Output:



```
1 class Book:
2     def __init__(self, title: str, author: str, isbn: str):
3         """Initialize a Book with title, author, ISBN, and
4           availability status."""
5         pass
6
7 class Member:
8     def __init__(self, name: str, member_id: int):
9         """Initialize a Member with name, ID, and a list of
10            borrowed books."""
11        pass
12
13 class Library:
14     def __init__(self):
15         """Initialize a Library with collections of books and
16            members."""
17        pass
18
19     def add_book(self, book: Book):
20         """Add a book to the library collection."""
21        pass
22
23     def add_member(self, member: Member):
24         """Register a new library member."""
25        pass
26
27     def borrow_book(self, member_id: int, isbn: str):
28         """Allow a member to borrow a book if available."""
29        pass
30
31     def return_book(self, isbn: str):
32         """Allow a member to return a borrowed book."""
33        pass
34
35     def display_available_books(self):
36         """Display all books that are currently available."""
37        pass
```

Case Study: Step 2 - Implementing Book and Member

- Focus: Implement the **simpler classes** first (Bottom-Up aspect within the structure).

- Example Prompt:

```
1 class Book:
2     # AI: Implement the __init__ method to store title, author,
3     # isbn.
4     # Initialize is_borrowed to False. Add a __repr__ method for
5     # easy printing.
6     def __init__(self, title: str, author: str, isbn: str):
7         """Initialize a Book with title, author, ISBN, and
8         availability status."""
9         pass
10
11 class Member:
12     # AI: Implement the __init__ method for name, member_id.
13     # Initialize borrowed_books as empty list.
14     # Add a __repr__ method.
15     def __init__(self, name: str, member_id: int):
16         """Initialize a Member with name, ID, and a list of
17         borrowed books."""
18         pass
```

[Provide Code]

Please complete the `__init__` methods in `Book` and `Member` classes according to my comments.

- Potential AI Output:

```
1 class Book:
2     def __init__(self, title: str, author: str, isbn: str):
3         """Initialize a Book with title, author, ISBN, and
4         availability status."""
5         self.title = title
6         self.author = author
7         self.isbn = isbn
8         self.is_borrowed = False # Initially, the book is
9         available
10
11     def __repr__(self):
12         """String representation for easy printing."""
13         status = "Borrowed" if self.is_borrowed else "Available"
14         return f"Book('{self.title}', '{self.author}', '{self.isbn}'
15                 ', {status})"
16
17 class Member:
18     def __init__(self, name: str, member_id: int):
19         """Initialize a Member with name, ID, and a list of
20         borrowed books."""
21         self.name = name
22         self.member_id = member_id
23         self.borrowed_books = [] # Stores books borrowed by the
24         member
25
26     def __repr__(self):
27         """String representation for easy printing."""
28         return f"Member('{self.name}', ID: {self.member_id},
29                 Borrowed books: {len(self.borrowed_books)})"
```

Case Study: Step 3 - Implementing Library Logic

- ◆ Challenge: Library methods involve more logic (finding books/members, checking conditions).
- ◆ Review: Carefully review the logic generated by AI, especially checks and state changes.
- ◆ Example Prompt:

```
30     def add_book(self, book: Book):
31         """Add a book to the library collection."""
32         # AI: Implement this method. It should take a Book object.
33         # Check if a book with the same ISBN already exists. If
           not, add the book to a list of books managed by the
           library (e.g., self.books). Print a confirmation or error
           message.
34         pass
```

[Provide Code]

Please complete the `add_book` methods in the `Library` class according to my comments.

◆ Potential AI Output:

```
31     def add_book(self, book: Book):
32         """Add a book to the library collection."""
33         if book.isbn in self.books:
34             print(f"Error: Book with ISBN {book.isbn} already exists.")
35         else:
36             self.books[book.isbn] = book
37             print(f"Book '{book.title}' added to the library.")
```



Case Study: Step 3 - Implementing Library Logic

- Challenge: Library methods involve more logic (finding books/members, checking conditions).
- Review: Carefully review the logic generated by AI, especially checks and state changes.
- Example Prompt:

```
40 def borrow_book(self, member_id: int, isbn: str):
41     """Allow a member to borrow a book if available."""
42     # AI: Implement borrowing. Find the member by member_id.
43     # Find the book by isbn.
44     # Check if member exists, book exists, and book is not
45     # already borrowed. If all ok, set book's is_borrowed to
46     # True, add book to member's borrowed_books list. Print
47     # status.
48     pass
```

[Provide Code]

Please complete the `borrow_book` methods in the `Library` class according to my comments.

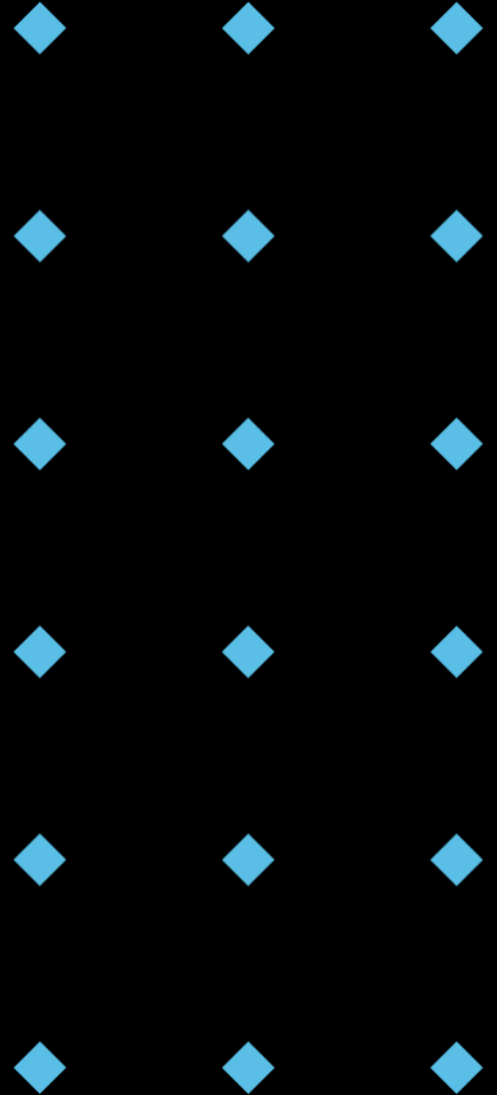
Potential AI Output:

```
47 def borrow_book(self, member_id: int, isbn: str):
48     """Allow a member to borrow a book if available."""
49     if member_id not in self.members:
50         print(f"Error: Member with ID {member_id} not found.")
51         return
52     if isbn not in self.books:
53         print(f"Error: Book with ISBN {isbn} not found.")
54         return
55
56     member = self.members[member_id]
57     book = self.books[isbn]
58
59     if book.is_borrowed:
60         print(f"Sorry, '{book.title}' is already borrowed.")
61     else:
62         book.is_borrowed = True
63         member.borrowed_books.append(book)
64         print(f"{member.name} borrowed '{book.title}'.")
```

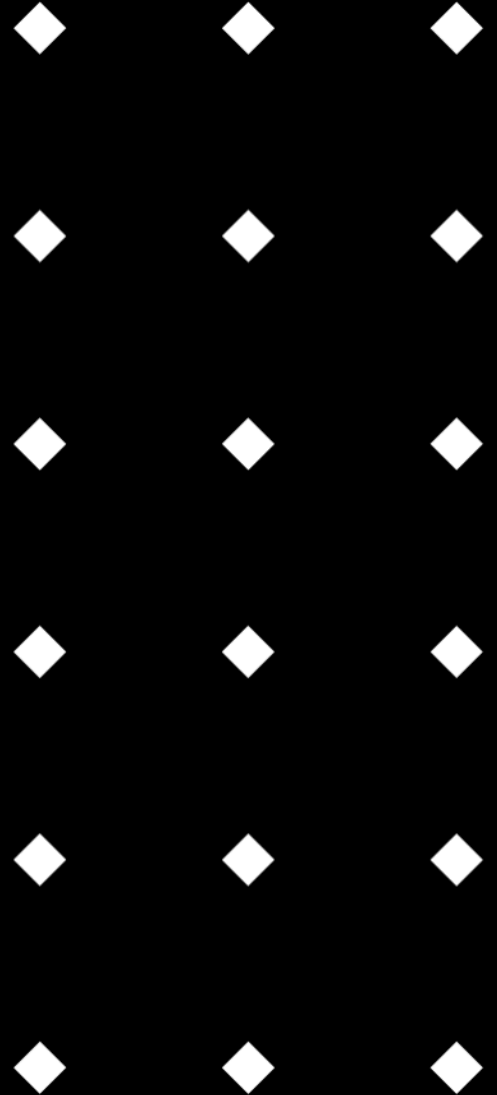
Case Study: Step 4 - Refinement and Testing

AI generated a first draft, now the developer takes over:

- ♦ **Review Logic:** Read through all generated code. Does it make sense? Is the logic correct? (e.g., does `return_book` correctly update `is_borrowed` and the member's list?).
- ♦ **Add Edge Case Handling:** What if `borrow_book` is called with an invalid `member_id`? What if the book ISBN doesn't exist? Did the AI handle this gracefully? Add checks if needed. (Will cover in Session 9)
- ♦ **Write Unit Tests:** Use `unittest` or `pytest` to create tests for each method. (Will cover in Session 9)
- ♦ **Refactor:** Improve code structure, clarity, variable names if needed.

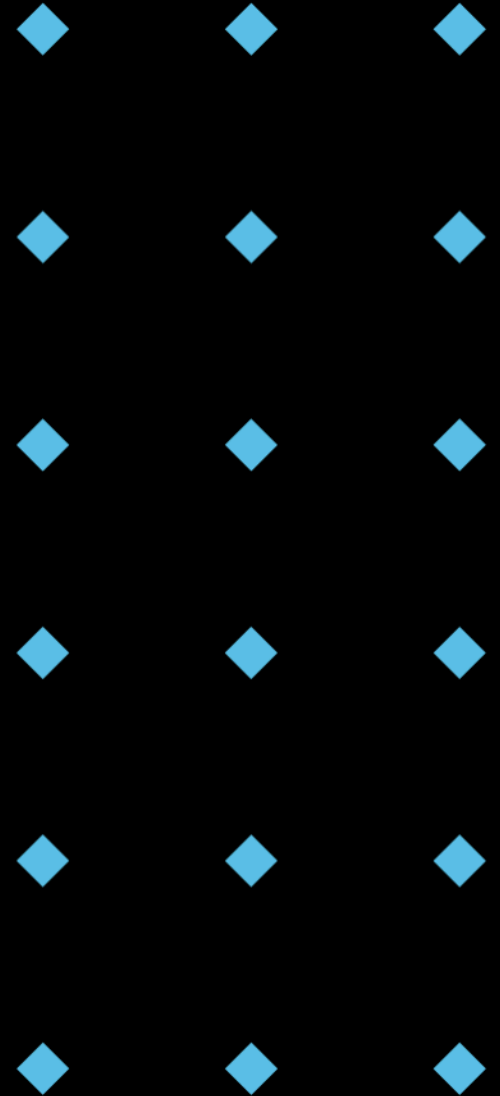


AI Assisted Debugging



AI-Assisted Debugging in OOP

- ◆ **Debugging:** The process of **finding** and **resolving** defects or problems ("bugs") within computer programs that prevent correct operation.
- ◆ Challenges in OOP Debugging:
 - ◆ State management across objects.
 - ◆ Complex interactions and dependencies.
 - ◆ Inheritance hierarchies (where does this method come from?).
 - ◆ Polymorphism (which version of the method is actually running?).
- ◆ AI's Role: Analyse error messages, code snippets, and natural language descriptions of problems to suggest **causes** and **fixes**.



Understanding Error Messages with AI

- ◆ Common Scenario: Python interpreter throws an error (e.g., *TypeError*, *AttributeError*, *NameError*, *IndexError*) with a traceback.
- ◆ How AI Helps:
 - ◆ **Paste the Error:** Copy the full error message and traceback into the AI tool.
 - ◆ Ask for **Explanation:** "Explain this Python error and traceback."
 - ◆ Ask for **Cause:** "What likely caused this error in my code context?" (May need to provide the relevant code snippet).
 - ◆ Ask for **Solution:** "How can I fix this error?"

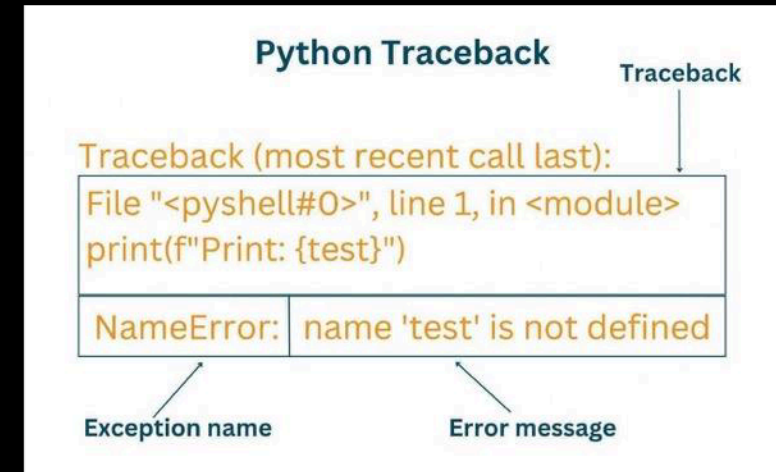


Image from Kusum Jain, Apr 12, 2023, <https://favtutor.com/blogs/python-traceback>

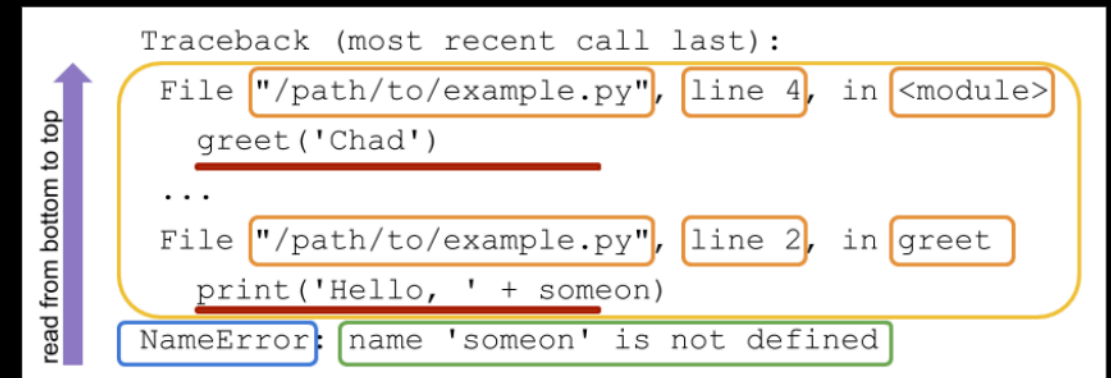


Image by Chad Hansen, 2019, <https://realpython.com/python-traceback/>

Example: Debugging AttributeError with AI

AI Interaction

[Provide Code]

Explain this error at line 7 of the above code: `AttributeError: 'Point' object has no attribute 'x'` from the line `print(p.x)`

This error means you are trying to access an attribute named `x` on an object of type `Point`, but the `Point` class (as defined or used here) does not have an attribute with that exact name. Check the object's available attributes or for typos.



How to fix?

Based on the likely definition of a `Point` class, you might have intended to access `p.x_coord`. Verify the attribute names defined in your `Point` class's `__init__` method.



```
1 class Point:
2     def __init__(self, x, y):
3         self.x_coord = x # Note the attribute name
4         self.y_coord = y
5
6 p = Point(10, 20)
7 print(p.x) # Typo! Should be p.x_coord
```

Traceback (most recent call last):

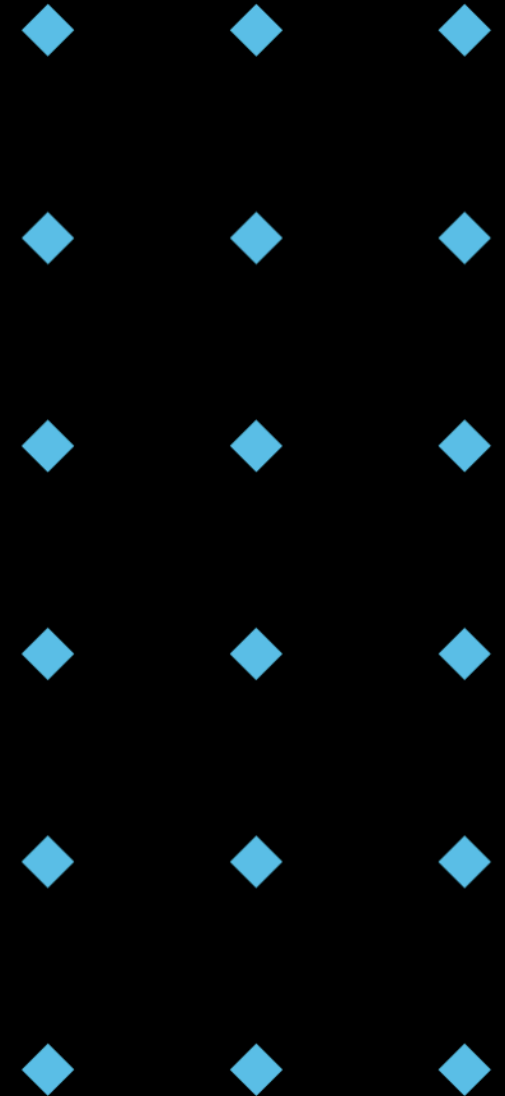
```
File "/Users/chenhaoxu/Downloads/test.py", line 7, in <module>
    print(p.x)
```

^^^

`AttributeError: 'Point' object has no attribute 'x'`

Beyond Syntax: Fixing Runtime/Logical Errors

- ◆ **Runtime Errors:** Code runs without crashing initially but produces incorrect results or behaves unexpectedly due to **flawed logic**. These are often **harder** to find than syntax errors or simple exceptions.
- ◆ How AI Can Help (with detailed context):
 - ◆ **Describe the Problem:** Explain the intended behaviour and the actual, incorrect behaviour.
 - ◆ **Provide Code:** Supply the relevant class(es) and the code section where the issue occurs.
 - ◆ **Ask for Analysis:** "Why might this function be returning the wrong value?" or "What could cause this object's state to be incorrect here?"
 - ◆ **AI May Suggest:** Potential logic flaws, boundary condition errors, incorrect state updates, race conditions (in concurrent code), etc.
- ◆ **Limitation:** AI **doesn't** run your code; its analysis is based on **patterns** and the **information** you provide. It can't magically find all logic bugs.



Runtime Debugging Example

```
1 class ShoppingCart:
2     def __init__(self):
3         self.items = [] # List of (item_name, price) tuples
4
5     def add_item(self, name, price):
6         if price > 0:
7             self.items.append((name, price))
8
9     def get_total_price(self):
10        total = 0
11        for item in self.items:
12            # Buggy line: item is ('item_name', price)
13            total += item # Should be total += item[1]
14        return total
```

- ◆ Problem: `get_total_price()` causes a *TypeError: unsupported operand type(s) for +=: 'int' and 'tuple'*.
- ◆ Question: How would you phrase a prompt to an AI to help you understand and fix this specific *TypeError* within the `get_total_price` method?

Runtime Debugging Example – Good Prompt

◆ Good Prompt:

I have this Python method in my `ShoppingCart` class. The `self.items` attribute is a list of tuples, where each tuple is `(item_name, price)`.

```
'''
def get_total_price(self):
    total = 0
    for item in self.items:
        total += item # This line causes an error
    return total
'''
```

When I run this method, I get the following error:
"TypeError: unsupported operand type(s) for +=: 'int' and 'tuple'"

1. Explain why this `TypeError` is occurring on the line `total += item`.
2. Show me the corrected code for the `get_total_price` method so that it correctly sums the prices from the tuples in `self.items`.

◆ Potential AI Output:



1. Why Does This `TypeError` Occur?

- `self.items` is a list of tuples where each tuple contains `(item_name, price)`, e.g.:

python

Copy Edit

```
self.items = [("apple", 1.5), ("banana", 2.0)]
```

- `item` in the loop is a tuple, e.g., `("apple", 1.5)`.
- `total` is an integer, initialized to `0`.
- The expression `total += item` tries to add an integer (`0`) to a tuple `(("apple", 1.5))`, which is **not supported** in Python.

2. Corrected Code

To correctly sum the prices, we need to access only the second element (`price`) from each tuple:

python

Copy Edit

```
def get_total_price(self):
    total = 0
    for item in self.items:
        total += item[1] # ✅ Access the price (second element of tuple)
    return total
```


Runtime Debugging Example – Bad Prompt

◆ Bad Prompt:

I get the following error:

"TypeError: unsupported operand type(s) for +=: 'int' and 'tuple'"

- 1. Explain why this `TypeError` is occurring on the line `total += item`.*
- 2. Show me the corrected code for the `get_total_price` method so that it correctly sums the prices from the tuples in `self.items`.*

◆ Potential AI Output:

Why This Error Occurs:

- If `self.items` is a list of tuples where each tuple contains item details (e.g., `("apple", 2, 3.5)` where `(name, quantity, price_per_unit)`), then `item` is assigned an entire tuple.
- When `total += item` is executed, Python tries to add an integer (`total`) to a tuple (`item`), which is not a valid operation.

Corrected Code:

To correctly sum the total price, you need to extract the price-related values from the tuple.

Assuming each tuple follows the format `(name, quantity, price_per_unit)`, you should compute the total price using:

python

Copy

Edit

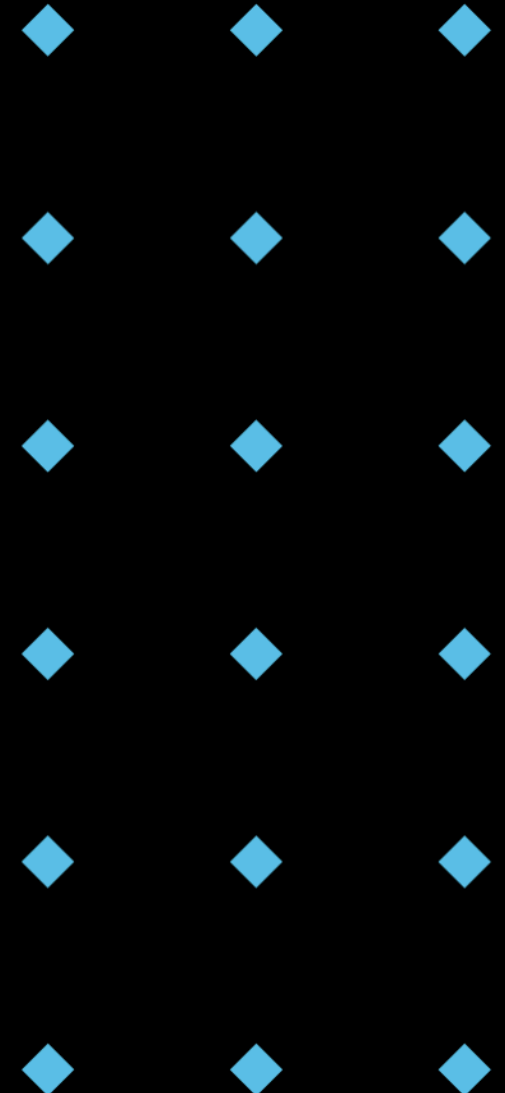
```
class ShoppingCart:
    def __init__(self):
        self.items = [] # List of tuples (name, quantity, price_per_unit)

    def add_item(self, name, quantity, price_per_unit):
        self.items.append((name, quantity, price_per_unit))

    def get_total_price(self):
        total = 0
        for item in self.items:
            _, quantity, price_per_unit = item # Unpacking tuple
            total += quantity * price_per_unit # Calculate total price for item
        return total
```

AI for Inheritance & Polymorphism Bugs

- ◆ Common Issues:
 - ◆ **Method Resolution Order (MRO):** In multiple inheritance, which parent method is called? (*ClassName.mro()* helps, but AI can explain).
 - ◆ **Accidental Overriding:** A subclass method unintentionally replaces a parent method.
 - ◆ ***super()* Usage:** Incorrectly calling (or forgetting to call) parent methods using *super()*.
 - ◆ **Polymorphic Behaviour:** Understanding why a specific subclass's method is being executed when working with a list of base class type objects.
- ◆ How AI Helps:
 - ◆ **Explain MRO:** "Explain the MRO for this class [provide class definitions with inheritance]."
 - ◆ **Trace Method Calls:** "If I call *obj.method()*, where *obj* is an instance of *SubClass*, which implementation of method gets called based on this inheritance structure [provide code]?"
 - ◆ **Analyse *super()*:** "Is *super().__init__()* used correctly in this subclass?"
 - ◆ **Explain Polymorphism:** "Explain why *item.draw()* calls different methods for *Circle* and *Square* objects in this loop [provide code]."



Example: Explaining MRO with AI

```
1 class A:
2     def ping(self): print("Ping from A")
3 class B(A):
4     def ping(self): print("Ping from B"); super().ping()
5 class C(A):
6     def ping(self): print("Ping from C"); super().ping()
7 class D(B, C):
8     def ping(self): print("Ping from D"); super().ping()
9
10 d = D()
11 d.ping()
12 # print(D.mro()) # Shows the order
```

◆ Example Prompt:

[Provide Code]

Explain the order of execution when `d.ping()` is called in this Python code, considering the multiple inheritance and `super()` calls. What is the MRO for class `D`?

◆ Potential AI Output:

✅ MRO (Method Resolution Order) for Class `D`

Python uses C3 Linearization to compute MRO. You can view the MRO like this:

python

Copy

Edit

```
print(D.__mro__)
```

```
# Output: (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

So the MRO is:

css

Copy

Edit

D → B → C → A → object

Important Caveats: Limitations & Ethics

- ◆ **Over-Reliance:** Don't let AI do your thinking! Understand the code it generates. Blindly copying can lead to subtle bugs or inefficient solutions.
- ◆ **Code Ownership & Plagiarism:** Understand the terms of use of your AI tool. Generated code might be based on existing codebases (potential licensing issues, though often mitigated by tools). Academic integrity requires citing AI assistance appropriately.
- ◆ **Security Vulnerabilities:** AI might inadvertently suggest code with security flaws (e.g., SQL injection, improper input validation) if not prompted carefully or if trained on insecure code.
- ◆ **Bias & Hallucination:** AI models can reflect biases in their training data or "hallucinate" plausible but incorrect information or code.
- ◆ **Context is King:** AI suggestions are only as good as the context provided. It might not understand the larger system architecture or specific constraints.
- ◆ **Debugging Still Requires Skill:** AI assists, but debugging complex issues still requires systematic investigation, hypothesis testing, and understanding the runtime behaviour – skills you need to develop.

Key Takeaways

AI for Coding:

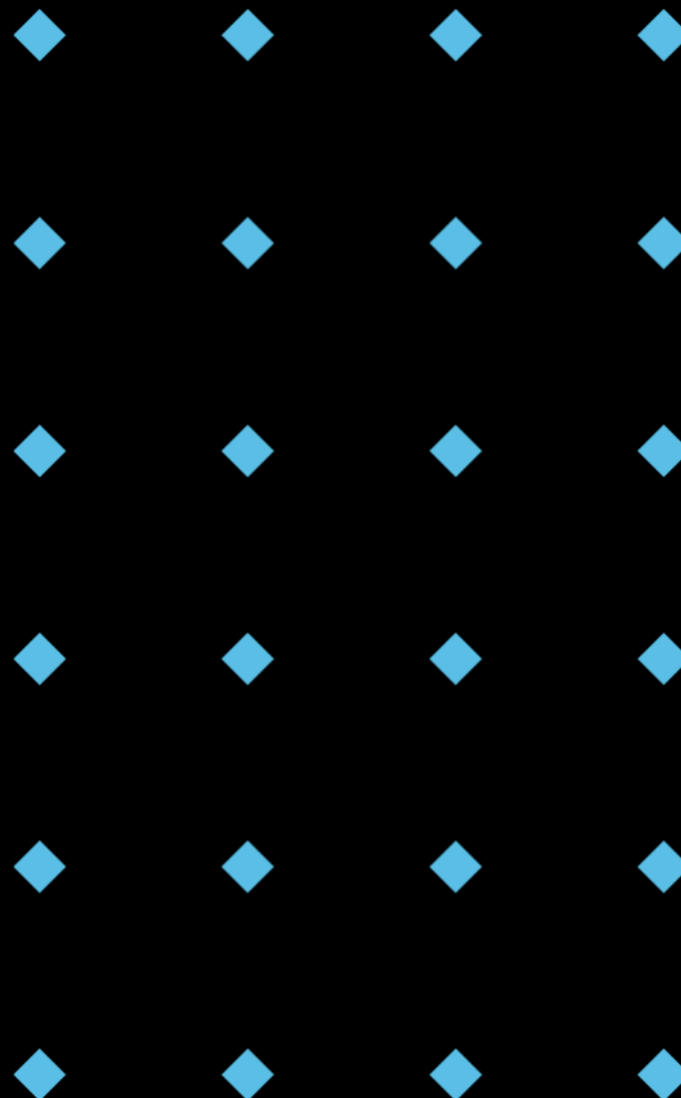
- ◆ Scaffolding: Quickly creates class skeletons (`__init__`, methods).
- ◆ Implementation: Generates method bodies from docstrings/comments.
- ◆ UML Conversion: Translates (textual descriptions of) UML diagrams to code.
- ◆ Design Support: Assists with both Top-Down and Bottom-Up approaches.

AI for Debugging:

- ◆ Error Analysis: Explains tracebacks and error messages.
- ◆ Runtime Logic: Suggests causes for unexpected behaviour (with context).
- ◆ Inheritance/Poly: Helps clarify MRO, `super()`, polymorphic calls.

Key Principle: AI Augmentation, Not Replacement. Review, Test, Understand!

Q & A



Lab

